

Parametry i argumenty funkcji

Funkcje z parametrami

Parametry to zmienne zdefiniowane w deklaracji funkcji, które działają jak „pojemniki” na dane, jakie funkcja może przyjąć. Podczas tworzenia funkcji, określamy, jakie dane będzie ona potrzebować do poprawnego działania, nadając tym zmiennym nazwy.

Argumenty to konkretne wartości przekazywane do funkcji w momencie jej wywołania. Są one przypisywane do odpowiednich parametrów funkcji. Dzięki temu funkcja może działać na różnych danych w zależności od tego, jakie argumenty zostały do niej przekazane.

Przykład:

```
function dodaj(a, b) { // 'a' i 'b' to parametry
  return a + b;
}

console.log(dodaj(3, 5)); // '3' i '5' to argumenty
```

W powyższym przykładzie funkcja `dodaj()` ma dwa parametry (`a` i `b`). Podczas jej wywołania z argumentami 3 i 5, wartości te są przypisane do parametrów, a funkcja zwraca ich sumę.

Wywołanie funkcji z argumentami jest kluczowym etapem jej działania, ponieważ dopiero wtedy funkcja przetwarza dane i wykonuje swoje zadanie.

Parametry określają „interfejs” funkcji, natomiast argumenty dostarczają funkcji rzeczywiste dane wejściowe.

Domyślne wartości parametrów

W JavaScript można przypisać domyślną wartość parametrom funkcji, co oznacza, że jeśli podczas wywołania funkcji argument nie zostanie przekazany, funkcja automatycznie użyje wartości domyślnej. Jest to szczególnie przydatne w sytuacjach, gdy nie zawsze wszystkie argumenty są wymagane.

Przykład:

```
function powitaj(imie = "Nieznajomy") {
  console.log(`Cześć, ${imie}!`);
}

powitaj("Łukasz"); // Wyjście: Cześć, Łukasz!
powitaj();         // Wyjście: Cześć, Nieznajomy!
```

W tym przykładzie parametr `imie` ma przypisaną wartość domyślną `"Nieznajomy"`. Jeśli podczas wywołania funkcji nie zostanie przekazany argument, funkcja automatycznie użyje tej wartości.

Domyślne wartości parametrów mogą być również oparte na wyniku wyrażenia:

Przykład:

```
function obliczSume(a, b = a * 2) {  
  return a + b;  
}  
  
console.log(obliczSume(5)); // Wyjście: 15 (b = 5 * 2)  
console.log(obliczSume(3, 7)); // Wyjście: 10 (b = 7)
```

W powyższym przykładzie wartość domyślna parametru `b` jest obliczana na podstawie wartości innego parametru `a`.

Dzięki domyślnym wartościom parametrów można tworzyć bardziej elastyczne i łatwe w użyciu funkcje, które lepiej radzą sobie z różnymi scenariuszami użycia.

Operator rest (...rest)

Operator rest (...rest) w JavaScript pozwala funkcji przyjmować dowolną liczbę argumentów i grupować je w jedną tablicę. Jest to szczególnie przydatne, gdy nie wiadomo z góry, ile argumentów zostanie przekazanych do funkcji, lub gdy chcemy pracować z dynamiczną liczbą danych wejściowych.

Jak działa operator rest?

Gdy operator rest jest używany jako ostatni parametr w deklaracji funkcji, zbiera wszystkie „pozostałe” argumenty, które nie zostały przypisane do wcześniejszych parametrów, i przechowuje je w tablicy.

Przykład:

```
function suma(...liczby) {  
  return liczby.reduce((acc, num) => acc + num, 0);  
}  
  
console.log(suma(1, 2, 3, 4, 5)); // Wyjście: 15  
console.log(suma(10, 20)); // Wyjście: 30
```

W tym przykładzie operator rest `...liczby` przechowuje wszystkie argumenty funkcji w tablicy `liczby`, a funkcja oblicza ich sumę przy użyciu metody `reduce`.

Metoda `reduce()` działa na tablicy i pozwala przekształcić ją w pojedynczą wartość.

Składnia:

```
array.reduce((accumulator, currentValue) => { /* operacja */ }, initialValue);
```

- **acc (accumulator):** To wartość, która przechowuje wynik obliczeń podczas iteracji. Zaczyna się od wartości początkowej, czyli `initialValue` (w tym przypadku 0).
- **num (currentValue):** To aktualny element tablicy podczas iteracji.

Zasady użycia

1. Musi być ostatnim parametrem funkcji:

```
function funkcjaNiepoprawna(a, ...reszta, b) {  
    // Błąd składniowy!  
}
```

2. Przechowuje dowolną liczbę argumentów w tablicy:

```
function wypiszElementy(...elementy) {  
    elementy.forEach((el, index) => {  
        console.log(`Element ${index + 1}: ${el}`);  
    });  
}  
  
wypiszElementy("jabłko", "banan", "gruszka");  
// Wyjście:  
// Element 1: jabłko  
// Element 2: banan  
// Element 3: gruszka
```

Wewnątrz metody forEach wykonywane jest:

```
console.log(`Element ${index + 1}: ${el}`);
```

- `index + 1`: Ponieważ indeksy w tablicy zaczynają się od 0, dodanie 1 powoduje, że elementy są numerowane od 1.
- `${el}`: To aktualny element tablicy, który jest wstawiany w miejsce zmiennej dzięki template string (`${}`).

Operator rest w połączeniu z innymi parametrami

Możesz łączyć operator rest z innymi parametrami, które są wymienione wcześniej w deklaracji funkcji.

Przykład:

```
function przywitajPierwszych(imiePierwsze, ...pozostali) {  
    console.log(`Cześć, ${imiePierwsze}!`);  
    console.log(`Pozostali: ${pozostali.join(", ")}`);  
}  
  
przywitajPierwszych("Ania", "Bartek", "Czarek", "Daria");  
// Wyjście:  
// Cześć, Ania!  
// Pozostali: Bartek, Czarek, Daria
```

W tym przypadku pierwszy argument jest przypisywany do `imiePierwsze`, a pozostałe są grupowane w tablicy `pozostali`.

Przydatne zastosowania

1. **Dynamiczna liczba argumentów:** Operator rest idealnie nadaje się do funkcji takich jak `Math.max`, gdy chcemy przekazać zmienną liczbę wartości:

```
function znajdzNajwieksza(...liczby) {  
  return Math.max(...liczby);  
}  
  
console.log(znajdzNajwieksza(10, 20, 30, 40)); // Wyjście: 40
```

2. **Tworzenie kopii tablic z dodatkowymi elementami:**

```
function dodajElementDoTablicy(pierwszy, ...reszta) {  
  return [pierwszy, ...reszta];  
}  
  
console.log(dodajElementDoTablicy("start", "a", "b", "c"));  
// Wyjście: ["start", "a", "b", "c"]
```

Operator rest pozwala uprościć pracę z argumentami i czyni funkcje bardziej elastycznymi oraz łatwiejszymi w obsłudze, co jest szczególnie przydatne w aplikacjach pracujących z dużymi zbiorami danych lub dynamicznymi parametrami.